

Enabling Feature Plane-based 3D Gaussian Splatting on Mobile VR from Video Streams

Soonbin Lee

Simon Sasse

Yago Sanchez

Cornelius Hellge

Thomas Schierl

Fraunhofer Heinrich-Hertz-Institute (HHI), Germany

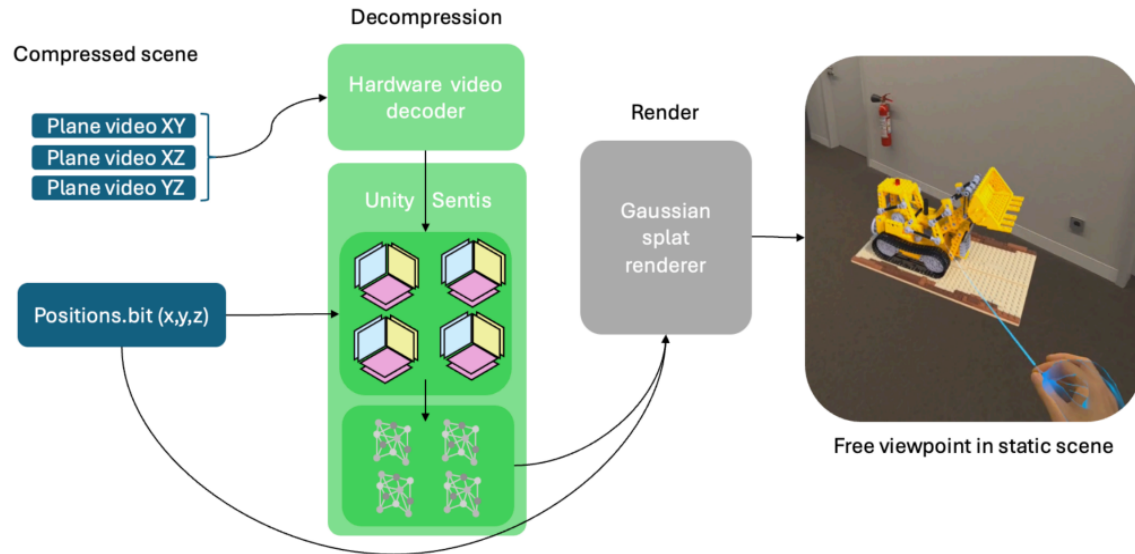


Figure 1: **Overview of the proposed CodecGS [15] decoding pipeline implemented in Unity.** The system utilizes hardware-accelerated video decoding to decompress frame-packed feature planes, which are then processed using Unity Sentis for MLP inference. The 3DGS scene is rendered on a Meta Quest 3 with Unity-based Rasterizer.

ABSTRACT

3D Gaussian Splatting (3DGS) has revolutionized real-time rendering capabilities, yet its substantial storage footprint significantly hinders deployment on resource-constrained mobile devices. While implicit representations like CodecGS effectively reduce storage by projecting Gaussian attributes onto feature planes, this framework relies on 16-bit data formats, rendering it incompatible with standard mobile hardware video decoders. This paper presents a practical, low-latency decoding pipeline for CodecGS, specifically optimized for the Meta Quest 3 within the Unity engine. We bridge the compatibility gap by integrating Unity's native FFmpeg for hardware-accelerated decoding and the Unity Sentis engine for on-device MLP inference. Crucially, we propose a strategic shift to 8-bit quantization for strict compatibility with mobile hardware constraints. By leveraging this pipeline, this paper demonstrates the end-to-end workflow for streaming and rendering CodecGS-based content on standalone VR headsets.

Index Terms: I.3.7 [Computer Graphics]: Three-Dimensional Graphics and Realism—Virtual reality; I.3.3 [Computer Graphics]: Picture/Image Generation—Display algorithms

1 INTRODUCTION

Standalone VR devices, such as the Meta Quest 3, operate under strict constraints regarding storage capacity and memory bandwidth. While 3D Gaussian Splatting (3DGS) [11] has emerged as a superior rendering technique, its explicit representation typically requires gigabytes of data, posing a significant bottleneck to mobile deployment. To address this scalability issue, CodecGS [15] introduced an implicit approach by projecting Gaussian attributes onto feature planes and compressing them using standard video codecs like HEVC [24]. This method effectively leverages spatial correlations within the scene to achieve high compression ratios. However, the transition from this research to standalone mobile VR presents a fundamental compatibility gap. The original CodecGS framework relies on 16-bit integer quantization to maintain high fidelity, necessitating the use of the HEVC Main 10 or Range Extension (RExt) profile [6, 10]. However, mobile Systems-on-Chip (SoCs), such as the Snapdragon XR2, do not support hardware acceleration for these 16-bit high-bit-depth profiles. This lack of native support renders the original CodecGS framework incompatible with standard mobile VR devices. Furthermore, existing implementations often depend on external libraries or offline reconstruction steps that are incompatible with the integrated runtime environment of game engines like Unity.

We present a practical decoding pipeline specifically adapted for the Unity engine to bridge this gap. We facilitate this by integrating Unity's native FFmpeg interface for feature plane decoding and the

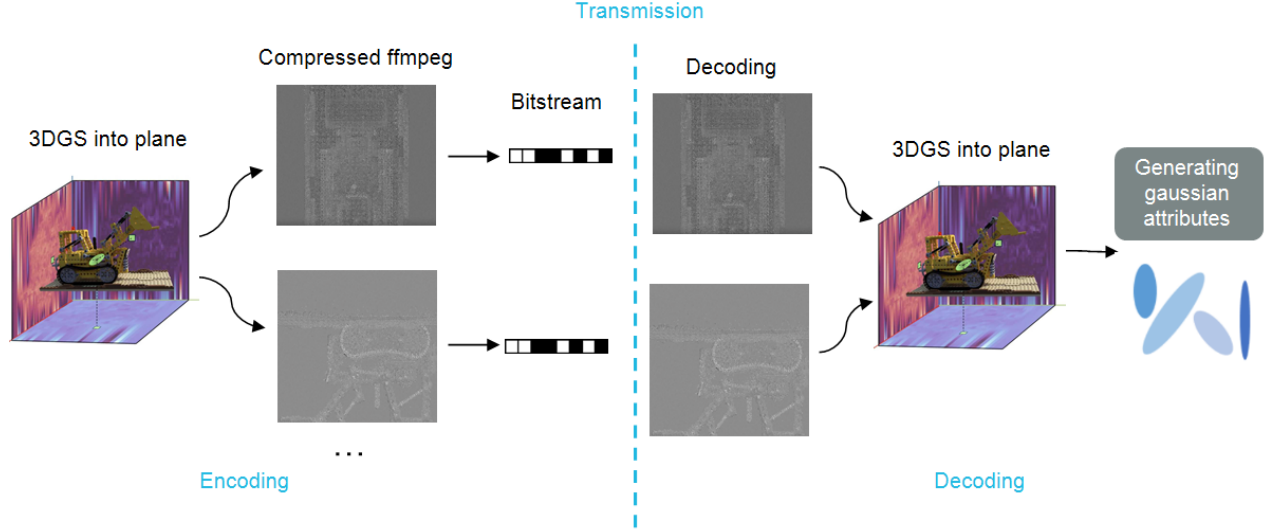


Figure 2: **Overview of the CodecGS data flow.** The encoding process projects 3D Gaussians onto feature planes and compresses them into ffmpeg video bitstreams. The decoding process utilizes hardware decoding and MLP inference to reconstruct the Gaussian attributes for rendering.

Unity Sentis engine for on-device MLP inference. Furthermore, by adopting an 8-bit quantization scheme, we overcome the hardware limitations of mobile SoCs, enabling a seamless on-device deployment that maintains a robust user experience. This Paper demonstrates the end-to-end workflow capable of loading and displaying CodecGS-based content directly on standalone VR headsets, proving the feasibility of video-based implicit representations for immersive mobile experiences.

2 RELATED WORK

Existing compression methods for 3D Gaussian Splatting generally fall into three categories [2]: pruning and quantization, entropy-based modeling, and video-based compression.

Pruning and Vector Quantization. A primary strategy to reduce storage is to eliminate redundant primitives and discretize attributes. Methods such as Compact3DGS [14] and LightGaussian [5] utilize importance-based pruning to remove Gaussians that contribute minimally to the rendered image. Furthermore, they employ Vector Quantization (VQ) to map gaussian attributes (e.g., color spherical harmonics and covariance matrices) into small codebooks [9, 21]. While effective in reducing model size, these explicit approaches often result in suboptimal rate-distortion performance compared to implicit representations. This is because they focus primarily on individual primitive reduction while neglecting the spatial redundancies and correlations inherent in the scene [1, 22, 25, 26].

Entropy Modeling with Arithmetic Coding. To achieve higher compression rates, recent methods like HAC [4] and Scaffold-GS [18] leverage entropy modeling with arithmetic coder [3, 8, 12, 16, 17]. Although these techniques achieve high compression ratios, they rely on complex, sequential decoding processes typically executed on the CPU [27]. This high decoding complexity imposes a significant bottleneck on mobile CPUs, making real-time rendering on standalone VR devices challenging without dedicated hardware acceleration.

Codec-based Attribute Compression. Another stream of research adapts standard 2D video codecs for 3DGS. Self-Organizing Gaussians (SOGS) [19] and similar works employ sorting algorithms (e.g., Morton order [13, 20]) to organize unstructured Gaussian attributes into grid-like pseudo-images for image compression. However, explicitly projecting millions of splats results in high-resolution

images that scale linearly with the splat count. CodecGS improved upon this by introducing an implicit Tri-plane representation [7], where attributes are predicted from compact feature planes compressed via video codecs. Our work builds directly on CodecGS, optimizing its decoding pipeline for mobile hardware by enforcing standard codec profiles and leveraging hardware acceleration to overcome the limitations of explicit and entropy modeling-based methods.

3 PROPOSED METHOD

We propose a mobile-optimized decoding pipeline for CodecGS, fully integrated into the Unity engine. By shifting to 8-bit quantization and utilizing Unity’s native FFmpeg interface, we resolve compatibility issues with mobile hardware decoders while ensuring real-time performance on standalone VR headsets. To ensure model compatibility and efficient on-device inference, we leverage ONNX and Unity Sentis. ONNX (Open Neural Network Exchange) serves as a standard bridge, allowing MLP models trained in Python environments to be exported for use in Unity. Unity Sentis directly loads ONNX models and executes them using the Meta Quest 3 without external dependencies. This integration effectively offloads computational tasks from the CPU, ensuring real-time rendering performance. For rendering, the reconstructed Gaussian attributes can be processed by any standard 3DGS rasterizer.

3.1 System Architecture

We demonstrate the proposed pipeline on a Meta Quest 3 headset. Our system adapts the CodecGS architecture for the Unity environment, executing the pipeline entirely on the VR client. The process begins with the HEVC-encoded video stream, which contains the packed feature planes P_{xy}, P_{xz}, P_{yz} . This stream is decoded into texture memory using Unity’s native FFmpeg interface. Following decoding, the feature planes are sampled based on the predetermined splat positions. These sampled features are then fed into an MLP decoder, implemented via Unity Sentis, to reconstruct the Gaussian attributes of color, scale, rotation, and opacity. Finally, the reconstructed attributes are passed to a Unity-based Gaussian rasterizer for display. Distinct from the feature planes, the Gaussian positions x, y, z are excluded from the video encoding pipeline in this work. This separation is primarily necessitated by the hardware

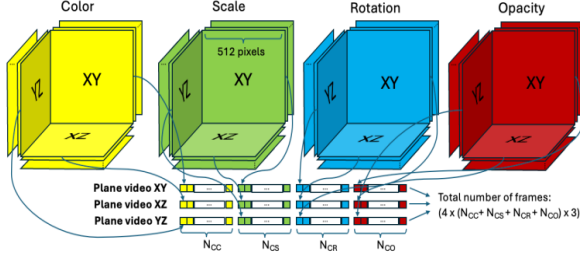


Figure 3: **Channel-to-frame packing for feature planes.** Gaussian attributes (Color, Scale, Rotation, Opacity) are decomposed into tri-planes (XY, XZ, YZ). To utilize standard video codecs, these multi-channel planes are spatially packed into video frames. Our implementation processes these packed frames to reconstruct the full attribute tensors.

constraints of mobile platforms, where standard video decoders and FFmpeg integrations often lack support for high level profiles. Since our pipeline adopts 8-bit quantization to ensure compatibility with the mobile hardware decoder, embedding position data into this low-precision stream would inevitably lead to severe artifacts. Therefore, we compress the position data using existing entropy coding tools and transmit it as a separate, high-precision bitstream.

To demonstrate the applicability of CodecGS, the decoding pipeline was implemented using Unity3D. As the target device, the Meta Quest 3 was selected due to its accessible Android-based platform and relatively large user base. The hardware video decoder is utilized to decode the feature planes, which are then further processed on the GPU. The Unity Sentis package provides a fast inference engine that facilitates this process. Additionally, it allows for the construction of simple models using common tensor operations provided by frameworks like PyTorch. The data required for the planes of the different attributes is distributed across the different plane videos. The tensor operations necessary for assigning the plane values to the correct models are implemented in a Unity Sentis model running on the GPU. The reconstructed attribute K-planes models are passed to another Unity Sentis model that combines the K-planes model and attribute MLPs into a single wrapper model. Inside this wrapper, the decoded data is passed to each attribute-specific feature plane, and the resulting output is processed by the MLP decoder with de-normalization.

3.2 Implementation Details

As illustrated in Figure 3, we employ a frame packing strategy to compress the high-dimensional feature representations into a standard video format. In this work, the scene is encoded into 8-channel feature planes with a resolution of 512×512 . These feature planes for each attribute, resulting in a total of 32 channels: Color N_{CC} , Scale N_{CS} , Rotation N_{CR} , and Opacity N_{CO} . These multi-channel feature planes are spatially tiled and concatenated into video frames. This layout allows the entire feature set to be treated as a standard HEVC video stream. To handle the HEVC bitstream on the Meta Quest 3, we utilize Unity’s native FFmpeg video decoding functions. Instead of using external media player plugins that typically incur context-switching overhead, we leverage Unity’s built-in FFmpeg wrappers. This integration ensures that the decoded video frames representing the feature planes are immediately available as Texture2D objects within the rendering thread. Consequently, this approach minimizes the latency between the decoding of a feature plane frame and its consumption by the inference engine, which is critical for maintaining high VR framerates.

A key modification from the original CodecGS is the adjustment

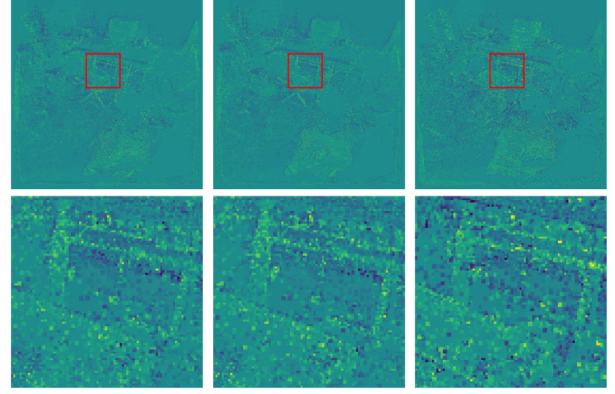


Figure 4: **Visualization of decoded feature planes.** The images display the first three channels of the feature planes from CodecGS.

of quantization depth. The original CodecGS utilizes 16-bit integer quantization to preserve the information of feature planes, a method that requires Main10 or specialized video profiles often unsupported on mobile hardware. The decision to downcast the feature planes from 16-bit to 8-bit quantization was driven by strict hardware constraints. While the original CodecGS implementation relies on high-fidelity 16-bit integer quantization, such data necessitates the use of the HEVC Main 10 profile. To convert the original 16-bit feature planes to 8-bit, we applied a per-plane min-max normalization. We stored the minimum and maximum values of each plane as metadata to de-normalize the attributes during the inference stage. This ensures that the dynamic range of the feature values is preserved within the 8-bit limit. On the Meta Quest 3, support for these high-bit-depth profiles within low-latency decoding contexts is often limited or incurs significant performance overhead. Since Unity does not provide a native Gaussian renderer, we integrated and adapted a third-party implementation by Aras Pranckevičius [23]. This renderer was modified to support the specific requirements of Head-Mounted Display (HMD) rendering. It is noted that the hardware limitations of the Meta Quest 3 currently allow for rendering primarily focused objects that do not occupy the full display area, typically not exceeding 50,000 to 200,000 Gaussian primitives for stable performance.

Table 1: Quantitative comparison of reconstruction quality and storage size between 16-bit and 8-bit quantization. Feature planes are configured with 8 channels and a resolution of 512×512 .

Quantization	PSNR $\uparrow$	LPIPS $\downarrow$	Plane Size (MB)
16-bit (Baseline)	31.67	0.190	6.62
8-bit (Ours)	31.21	0.196	6.55

4 EXPERIMENTAL RESULTS AND DEMO

We evaluated the proposed pipeline on a Meta Quest 3 headset to assess both reconstruction quality and runtime performance. To validate our mobile-optimized pipeline, we encoded the original feature planes using FFmpeg x265 with 8-bit quantization, targeting the standard Main profile. We first analyzed the impact of reducing quantization on reconstruction quality. Table 1 presents a comparison between the baseline 16-bit quantization and our proposed 8-bit quantization.

As shown in Table 1, switching to 8-bit quantization results in a slight decrease in PSNR from 31.67 dB to 31.21 dB and a marginal increase in LPIPS from 0.190 to 0.196 (Bonsai scene). Importantly,

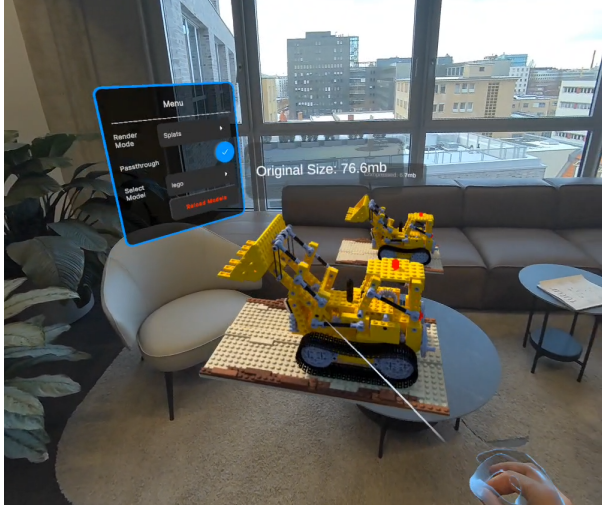


Figure 5: **Demonstration on Meta Quest 3.** The figure shows the full 3DGS scene rendered on the standalone VR headset with CodecGS decoding pipeline.

this modification allows the system to leverage standard hardware video decoders, which is a mandatory requirement for smooth execution on mobile VR devices. The reduction in plane size was marginal (6.62 MB vs. 6.55 MB). This is likely because the feature plane parameters were not optimized to align with the 8-bit quantization configuration, leading to suboptimal compression efficiency. Regarding visual quality, while the switch to 8-bit quantization introduced minor artifacts compared to the 16-bit reference implementations, the visual quality remained robust for immersive viewing. This validates the trade-off, demonstrating that the compatibility gained by 8-bit quantization is essential for mobile feasibility without compromising the user experience.

Figure 4 illustrates the visual output of the feature planes decoded directly on the mobile device. These textures, corresponding to the first three channels of the implicit attribute representation, are retrieved from the HEVC video stream following 8-bit quantization. Unlike standard RGB images, the pixel values encode feature plane parameters mapped onto the 2D spatial grid. As shown in the visualization, the spatial structure of these parameters is preserved despite the reduced bit depth. This confirms that our mobile-optimized pipeline successfully utilizes the Meta Quest 3’s hardware decoder to reconstruct the signal with sufficient fidelity, providing robust input for the subsequent MLP inference stage to recover the Gaussian attributes.

As shown in Figure 5, the proposed system achieves real-time performance on the Meta Quest 3, suitable for immersive VR experiences. We observed that the frame packing strategy effectively utilizes the hardware decoder, allowing millions of splats to be implicitly represented within a standard video resolution that is compliant with the device’s decoder level. Once the Gaussian attribute parameters are fully regenerated from the decoding pipeline, the rendering phase becomes identical to the original 3D Gaussian Splatting framework. This ensures that the system enables free-viewpoint exploration and supports standard rasterization techniques with the same versatility as uncompressed, explicit 3DGS representations.

We further analyzed the initialization and decoding latency. Table 2 provides a breakdown of the processing time for a single scene. The total initialization time is approximately 4.69 seconds. The breakdown reveals that MLP Inference (Sentis) is the most computationally intensive step, taking 2.74 seconds, whereas the

Table 2: Breakdown of initialization and decoding runtime for a scene on Meta Quest 3 (Number of Splats N is about 0.33M).

Processing Step	Time (s)
FFmpeg Video Decoding	0.78
Load Models (ONNX)	0.81
MLP Inference (Sentis)	2.74
Setup Renderer	0.36
Total Initialization Time	4.69

video decoding takes only 0.78 seconds. This demonstrates that our pipeline effectively leverages the hardware decoder to minimize loading times, although MLP inference remains a bottleneck to be optimized in future work. It is important to note that the total initialization time of 4.69 seconds represents a one-time loading cost per static scene. While the MLP inference accounts for the majority of this overhead (2.74s), this latency occurs only during the initial scene reconstruction phase and does not affect the runtime rendering framerate. The MLP decoder, executed via Unity Sentis, consists of 2 hidden layers with 128 units for each feature planes, utilizing ReLU activation functions. We specifically designed this shallow architecture to minimize the inference latency on the mobile GPU while preserving sufficient capacity to reconstruct the Gaussian attributes from the compressed feature planes.

5 DISCUSSION AND LIMITATIONS

While our current implementation successfully achieves interactive rendering performance, the initialization phase remains constrained by the computational overhead of the MLP decoder. As observed in our experiments, the inference step accounts for approximately 60% of the total loading time, creating a bottleneck for instant scene viewing. To address this, future iterations of this pipeline could adopt the explicit linear decoding strategy proposed in the K-Planes framework. By replacing the non-linear neural network layers with learnable basis matrices, the attribute reconstruction process is transformed into efficient linear matrix operations. This reduction in computational complexity is particularly advantageous for mobile GPUs, potentially minimizing the initialization latency significantly.

In addition, a promising direction is to employ a bit-splitting technique for Gaussian positions. By distributing the Most Significant Bits (MSB) and Least Significant Bits (LSB) of high-precision position values across separate 8-bit video channels, we can encode geometry within standard video profiles. This method encapsulates the full 3DGS representation into a single unified video stream, thereby simplifying the synchronization between geometry and attributes while fully leveraging the throughput of hardware video decoders. Consequently, this approach is expected to allow for the integration of compressed data into a unified video stream structure.

6 CONCLUSION

We presented a 3DGS Unity implementation of CodecGS tailored for Mobile VR. By adapting the Tri-plane architecture with Unity engine, and optimizing the data pipeline for 8-bit quantization, we successfully bridged the gap between high-fidelity research models and deployed VR hardware. This paper highlights the feasibility of using video-based feature planes for efficient 3D streaming on standalone devices, paving the way for more immersive and accessible 3D content in mobile virtual reality environments. Consequently, this approach bridges the gap between academic compression algorithms and practical application, facilitating the immediate deployment of photorealistic 3D content on standalone devices.

REFERENCES

- [1] M. S. Ali, M. Qamar, S.-H. Bae, and E. Tartaglione. Trimming the Fat: Efficient Compression of 3D Gaussian Splats through Pruning, 2024.
- [2] M. T. Bagdasarian, P. Knoll, Y. Li, F. Barthel, A. Hilsmann, P. Eisert, and W. Morgenstern. 3DGS.zip: A Survey on 3D Gaussian Splatting Compression Methods. *Computer Graphics Forum*, p. e70078, 2025. doi: 10.1111/cgf.70078
- [3] J. Ballé, P. A. Chou, D. Minnen, S. Singh, N. Johnston, E. Agustsson, S. J. Hwang, and G. Toderici. Nonlinear transform coding. *IEEE Journal of Selected Topics in Signal Processing*, 15(2):339–353, 2020.
- [4] Y. Chen, Q. Wu, W. Lin, M. Harandi, and J. Cai. Hac: Hash-grid assisted context for 3d gaussian splatting compression. In *European Conference on Computer Vision*, 2024.
- [5] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang. LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS. *arXiv preprint arXiv:2311.17245*, pp. 1–16, 2023.
- [6] <http://ffmpeg.org/>. FFmpeg Developers. Accessed: 2024-09-12.
- [7] S. Fridovich-Keil, G. Meanti, F. R. Warburg, B. Recht, and A. Kanazawa. K-planes: Explicit radiance fields in space, time, and appearance. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12479–12488, 2023.
- [8] S. Girish, K. Gupta, and A. Shrivastava. Eagles: Efficient accelerated 3d gaussians with lightweight encodings. In *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part LXIII*, p. 54–71. Springer-Verlag, Berlin, Heidelberg, 2024.
- [9] R. Gray. Vector quantization. *IEEE Assp Magazine*, 1(2):4–29, 1984.
- [10] u. ISO/IEC 23008-2 High Efficiency Video Coding (HEVC) Test Model reference software. Accessed: 2024-09-12.
- [11] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (ToG)*, 42(4):1–14, 2023.
- [12] G. G. Langdon. An introduction to arithmetic coding. *IBM Journal of Research and Development*, 28(2):135–149, 1984. doi: 10.1147/rld.282.0135
- [13] J. C. Lee, J. H. Ko, and E. Park. Optimized Minimal 3D Gaussian Splatting. *arXiv preprint:2503.16924*, 2025.
- [14] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park. Compact 3d gaussian representation for radiance field. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 21719–21728, 2024.
- [15] S. Lee, F. Shu, Y. Sanchez, T. Schierl, and C. Hellge. Compression of 3d gaussian splatting with optimized feature planes and standard video codecs. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 25496–25505, October 2025.
- [16] S. Lee, F. Shu, Y. Sánchez, T. Schierl, and C. Hellge. Ecrf: Entropy-constrained neural radiance fields compression with frequency domain optimization. In *2024 IEEE 26th International Workshop on Multimedia Signal Processing (MMSP)*, pp. 1–6, 2024. doi: 10.1109/MMSP61759.2024.10743749
- [17] X. Liu, X. Wu, P. Zhang, S. Wang, Z. Li, and S. Kwong. CompGS: Efficient 3D Scene Representation via Compressed Gaussian Splatting. In *Proceedings of the 32nd ACM International Conference on Multimedia, MM '24*, p. 2936–2944. Association for Computing Machinery, New York, NY, USA, 2024. doi: 10.1145/3664647.3681468
- [18] T. Lu, M. Yu, L. Xu, Y. Xiangli, L. Wang, D. Lin, and B. Dai. Scaffoldgs: Structured 3D Gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20654–20664, 2024.
- [19] W. Morgenstern, F. Barthel, A. Hilsmann, and P. Eisert. Compact 3d scene representation via self-organizing gaussian grids, 2024.
- [20] G. M. Morton. A computer oriented geodetic data base; and a new technique in file sequencing. Technical report, IBM Ltd., Ottawa, Canada, 1966.
- [21] K. Navaneet, K. P. Meibodi, S. A. Koohpayegani, and H. Pirsiavash. Compact3D: Compressing Gaussian Splat Radiance Field Models with Vector Quantization. *arXiv preprint arXiv:2311.18159*, pp. 1–12, 2023.
- [22] S. Niedermayr, J. Stumpfegger, and R. Westermann. Compressed 3d gaussian splatting for accelerated novel view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10349–10358, June 2024.
- [23] A. Pranckevičius. Gaussian splatting playground in Unity. <https://github.com/aras-p/UnityGaussianSplatting/releases/tag/v0.9.1>, 2024. Version 0.9.1.
- [24] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand. Overview of the high efficiency video coding (hevc) standard. *IEEE Transactions on circuits and systems for video technology (TCSVT)*, 22(12):1649–1668, 2012.
- [25] L. Wang, K. Yao, C. Guo, Z. Zhang, Q. Hu, J. Yu, L. Xu, and M. Wu. VideoRF: Rendering Dynamic Radiance Fields as 2D Feature Video Streams. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 470–481. IEEE Computer Society, Los Alamitos, CA, USA, June 2024. doi: 10.1109/CVPR52733.2024.00052
- [26] M. Wu, Z. Wang, G. Kourous, and T. Tuytelaars. TeTriRF: Temporal Tri-Plane Radiance Fields for Efficient Free-Viewpoint Video. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6487–6496. IEEE Computer Society, Los Alamitos, CA, USA, June 2024. doi: 10.1109/CVPR52733.2024.00620
- [27] Y. Zhan, C. Ho, H. Yang, Y. Chen, J. Chiang, Y. Liu, and W. Peng. CAT-3DGS: A context-adaptive triplane approach to rate-distortion-optimized 3dgs compression. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net, 2025.